By Hand: 15 min
10/15/1999—10/25/1999, 10/23/1999—12/1/1999, 10/21/1999—3/4/2004

1. Start date: 15, End date: 25, 25 - 16 = 9days.
2. Start date: 10/23, End date: 12/1, days left of Oct: 31 - 24 = 7 + days of Nov: 30 + Days into Dec: 1 = 38 days.
3. Start date: 10/21/1999, End date: 3/4/2004, days left of Oct: 31 - 22 = 9 + days of Nov: 30 + days of Dec: 31, 2000 is leap year: + 366, 2001 not leap: + 365, 2002 not leap: + 365, 2003 not leap: + 365, 2004 is leap, and end year: days in Jan: + 31, days in Frb/leap: + 29, days into last month: 4 = 9 + 30 + 31 + 366 + 365 + 365 + 365 + 31 + 29 + 4 = 1595 days.

Approach: 15 min
To execute this algorithm, we start by looking at the start and end dates. Regardless of what we are calculating for, we add one to the start date, this way we stay consistent to keep track of how many days are BETWEEN the start and end dates. If the years are different, complete the starting year and check again to see if they are the same. If not, check to see if it is a leap year and add a whole year until they are the same year. Once the years are the same, we will need to calculate the months; add the number of days in the month until the month dates are the same(make sure to keep an eye out for February, if it is a leap it will be 29 days, if not, it will be 28 days). Once the month dates are the same, add the end date to the total.

Pseudocode: 1 hour 30 min

```
GET start_day
GET start_month
GET start_year

GET end_day
GET end_month
GET end_year

days_between = 0

# Applies to both start and end.
# Using assert here, but for actual code, it isn't the right kind of
#check.
Assert 1 <= day <= 31
Assert 1 <= month <= 12
Assert 1753 <= year
Assert len(year) == 3 # (0-3)


Def days_calc
     IF all_end == all_start
          PUT 0 days between, same day

     start_day += 1
```

```
IF start_year == end_year
     IF start_month == end_month
          days_between = end_day - start_day
          return(days_between)

     IF start_month != end_month

          days_between += days_in_month - start_day
          start_month += 1

          IF start_month > 12
               start_month = 1
               start_year += 1

          WHILE start_month < end_month
               IF start_month == 2

                    IF is_leap_year(start_year)
                         days_between += 29
                    ELSE
                         days_between += 28

               ELSE IF start_month == 4 OR start_month == 6 OR
               start_month == 9 OR start_month == 11
                    days_between += 30

               ELSE
                    days_between += 31

               start_month += 1

          days_between += end_day
          return(days_between)


IF start_year != end year

     WHILE start_year < end_year

          IF is_leap_year(start_year)
               days_between += 366
          ELSE
               days_between += 365

          start_year += 1

     IF start_month == end_month
          days_between = end_day - start_day
          return(days_between)

     IF start_month != end_month
```

```
                IF start_month == 2

                      IF is_leap_year(start_year)
                            days_between += 29 - start_day
                      ELSE
                            days_between += 28 - start_day

                ELSE IF start_month == 4 OR start_month == 6 OR
                start_month == 9 OR start_month == 11
                      days_between += 30 - start_day

                ELSE
                      days_between += 31 - start_day

                start_month += 1

                IF start_month > 12
                      start_month = 1
                      start_year += 1

                WHILE start_month < end_month
                      days_between += days_in_month
                      start_month += 1

                      IF start_month > 12
                            start_month = 1
                            start_year += 1

                days_between += end_day
                return(days_between)


PUT days_calc
```

AI pseudocode: 7 min

```
INPUT startMonth, startDay, startYear
INPUT endMonth, endDay, endYear

totalDays ← 0

// Move one day past the start date
startDay ← startDay + 1

// Finish the starting month
IF startDay ≤ DaysInMonth(startMonth, startYear) THEN
    totalDays ← totalDays +
                 (DaysInMonth(startMonth, startYear) - startDay)
END IF

startMonth ← startMonth + 1
```

```
// Add remaining months in the starting year
WHILE startYear < endYear DO

    WHILE startMonth ≤ 12 DO
        totalDays ← totalDays +
                    DaysInMonth(startMonth, startYear)

        startMonth ← startMonth + 1
    END WHILE

    startYear ← startYear + 1
    startMonth ← 1

END WHILE

// Add months before the ending month
WHILE startMonth < endMonth DO

    totalDays ← totalDays +
                DaysInMonth(startMonth, startYear)

    startMonth ← startMonth + 1

END WHILE

// Add days into the ending month
totalDays ← totalDays + endDay

OUTPUT totalDays


FUNCTION DaysInMonth(month, year)

    IF month = 2 THEN
        IF year is a leap year THEN
            RETURN 29
        ELSE
            RETURN 28
        END IF

    ELSE IF month = 4 OR 6 OR 9 OR 11 THEN
        RETURN 30

    ELSE
        RETURN 31
    END IF

END FUNCTION
```

Compare and contrast: 10 min

Some of the pros of mine are that it is highly detailed, and is able to check for all the solutions, but it is really long, and a lot of the if statements could probably be simplified. AI's code uses functions to help simplify the equation, as well as being much shorter and easier to comprehend. Mine can use functions in order to simplify itself and stay shorter. I think that a lot of checks and loop cycles could be avoided in AI. I think that my code matches the by-hand example pretty well, as I can see where all the variables are being added, and it accounts for all the leap year exceptions. In AI's code, I don't think that it does it as well by hand, because it uses loops to add the numbers by the year, rather than adding the whole year, depending on whether it is a leap year or not.

Update: 28 min

```
GET start_day   # A
GET start_month # B
GET start_year  # C

GET end_day     # D
GET end_month   # E
GET end_year    # F

days_between = 0      # G

ASSERT 1 <= start_day <= 31
ASSERT 1 <= end_day <= 31

ASSERT 1 <= start_month <= 12
ASSERT 1 <= end_month <= 12

ASSERT start_year >= 1753
ASSERT start_year <= end_year

FUNCTION days_in_month(month, year)

IF month == 2

      IF is_leap_year(year)
            RETURN 29
      ELSE
            RETURN 28

ELSE IF month == 4 OR month == 6 OR month == 9 OR month == 11
      RETURN 30

ELSE
      RETURN 31

FUNCTION days_calc()

IF start_day == end_day AND                 #
      start_month == end_month AND    # H
      start_year == end_year          #

      RETURN 0
```

```
start_day += 1  # I

IF start_year == end_year
     IF start_month == end_month

     RETURN end_day - start_day


# Finish the starting month
days_between += days_in_month(start_month, start_year) - start_day #
J


start_month += 1          # K


IF start_month > 12
     start_month = 1 # L
     start_year += 1 # M


# Finish the starting year
WHILE start_year < end_year AND start_month <= 12

     days_between += days_in_month(start_month, start_year) # N

     start_month += 1          #O

     IF start_month > 12
          start_month = 1 #P
          start_year += 1 #Q


# Add complete years between start and end years
WHILE start_year < end_year

     IF is_leap_year(start_year)
          days_between += 366   # R
     ELSE
          days_between += 365   # S

     start_year += 1           # T


# Add months in the ending year
WHILE start_month < end_month

     days_between += days_in_month(start_month, start_year) # U

     start_month += 1          # V
```

```
# Add days into ending month
days_between += end_day    # W

RETURN days_between
PUT days_calc()            # X
```

Trace: 26 min

#	start_day	start_month	start_ year	end_day	end_ month	end_year	days_between
A - G	17	11	2002	6	4	2004	0
I	18	11	2002	6	4	2004	0
J	18	11	2002	6	4	2004	12
K	18	12	2002	6	4	2004	12
N	18	12	2002	6	4	2004	43
O	18	13	2002	6	4	2004	43
P	18	1	2002	6	4	2004	43
Q	18	1	2003	6	4	2004	43
S	18	1	2003	6	4	2004	408
T	18	1	2004	6	4	2004	408
U	18	1	2004	6	4	2004	439
V	18	2	2004	6	4	2004	439
U	18	2	2004	6	4	2004	468
V	18	3	2004	6	4	2004	468
U	18	3	2004	6	4	2004	499
V	18	4	2004	6	4	2004	499
W	18	4	2004	6	4	2004	505
X	18	4	2004	6	4	2004	505

Efficiency: 3 min

While there are multiple while loops, there isn't an actual nested loop anywhere, meaning that the efficiency should simply be O(n). It runs through the loop (n) times without ever multiplying the number of times it needs to run.